

CESAR SCHOOL — CENTRO DE ESTUDOS E SISTEMAS AVANÇADOS DO RECIFE

Refatoração de Condicionais via Lógica Clássica

Lógica Matemática para Computação (LMC) & Programação Imperativa e Funcional (PIF)

Data da Atividade: 23/09/2026 | **Projeto:** AI Safety — Alinhamento do Modelo AURA-67 (Squad E7) | **Linguagem:** C99 / Raylib**Integrantes da Equipe Presentes em Sala (23/09/2026):**

- | | |
|--|-----------------------------------|
| 1. João Gabriel Vitorino Dornelas (Responsável pelo Envio) | 2. Caio Alves Ferreira Brayner |
| 3. Matheus Henrique de Lucena Chaves | 4. Julio Cesar dos Santos Almeida |
| 5. Jhorge Alberto de Araujo | 6. Theo Monteiro Pinho |
| 7. Larissa Vitoria Silva de Almeida | 8. Mateus Almeida de Lacerda |

CONTEXTUALIZAÇÃO E APOIO COM INTELIGÊNCIA ARTIFICIAL

No desenvolvimento do jogo *AI Safety*, a equipe utilizou ferramenta de Inteligência Artificial para auditar expressões condicionais no código-fonte em C (arquivo `src/main.c`) que pudessem ser simplificadas por meio das leis da Lógica Clássica. A alteração foi analisada, revisada e comprovada formalmente pela squad, garantindo a preservação exata do comportamento lógico original do programa.

1 — CÓDIGO ORIGINAL (ANTES DA MELHORIA)

Na rotina de encerramento de partida, o sistema verificava a tecla de reinicialização em blocos `if / else if` separados para os estados de derrota e vitória, duplicando a leitura do teclado e as instruções de reinicialização:

```
// src/main.c - Trecho original redundante
if (state == STATE_GAME_OVER && IsKeyPressed(KEY_R)) {
    player_init(&player, SCREEN_WIDTH / 2.0f, SCREEN_HEIGHT - 200.0f);
    boss_init(&boss, SCREEN_WIDTH / 2.0f, 130.0f, difficulty);
    bullet_mgr_clear(&bullet_mgr);
    typing_engine_reset(&typing_engine);
    ui_init(&ui);
    score = 0;
    state = STATE_COMBAT;
} else if (state == STATE_VICTORY && IsKeyPressed(KEY_R)) {
    player_init(&player, SCREEN_WIDTH / 2.0f, SCREEN_HEIGHT - 200.0f);
    boss_init(&boss, SCREEN_WIDTH / 2.0f, 130.0f, difficulty);
    bullet_mgr_clear(&bullet_mgr);
    typing_engine_reset(&typing_engine);
    ui_init(&ui);
    score = 0;
    state = STATE_COMBAT;
}
```

2 — CÓDIGO OTIMIZADO (DEPOIS DA MELHORIA)

Aplicando a **Propriedade Distributiva da Conjunção sobre a Disjunção** (fatoração lógica do termo comum `IsKeyPressed(KEY_R)`), unificou-se a estrutura em um único bloco condicional:

```
// src/main.c - Trecho refatorado via Lógica Clássica
if ((state == STATE_GAME_OVER || state == STATE_VICTORY) && IsKeyPressed(KEY_R)) {
    player_init(&player, SCREEN_WIDTH / 2.0f, SCREEN_HEIGHT - 200.0f);
    boss_init(&boss, SCREEN_WIDTH / 2.0f, 130.0f, difficulty);
    bullet_mgr_clear(&bullet_mgr);
    typing_engine_reset(&typing_engine);
    ui_init(&ui);
    score = 0;
    state = STATE_COMBAT;
}
```

3 — ANÁLISE DA MELHORIA OBTIDA

A alteração produziu melhorias objetivas mensuráveis:

- **Operadores lógicos antes:** 2 conjunções (&&) e 1 desvio condicional else if.
- **Operadores lógicos depois:** 1 disjunção (||) e 1 conjunção (&&).
- **Chamadas de hardware//O:** A função IsKeyPressed(KEY_R) era avaliada em 2 pontos diferentes do código; na versão nova é avaliada apenas 1 vez.
- **Redução de redundâncias:** Eliminação de 8 linhas duplicadas de inicialização de variáveis e entidades.
- **Redução de linhas:** De 18 linhas para 10 linhas no bloco (redução de 44% de código).
- **Eficiência por curto-circuito:** Em C, se o jogo estiver em combate ativo (state == STATE_COMBAT), a disjunção (state == STATE_GAME_OVER || state == STATE_VICTORY) avalia para falso imediatamente, evitando qualquer leitura desnecessária do buffer de teclado durante o loop de 60 FPS.

4 — DEMONSTRAÇÃO DA FERRAMENTA DE LÓGICA UTILIZADA

Definição das Proposições:

- P: state == STATE_GAME_OVER (operador foi derrotado)
- Q: state == STATE_VICTORY (modelo AURA-67 foi alinhado com sucesso)
- R: IsKeyPressed(KEY_R) (tecla 'R' de reinício foi pressionada)

Expressão Lógica Original:

$$(P \wedge R) \vee (Q \wedge R)$$

Propriedade Aplicada:

Propriedade **Distributiva da Conjunção sobre a Disjunção** (fatoração proposicional):

$$(A \wedge C) \vee (B \wedge C) \equiv (A \vee B) \wedge C$$

Fatorando a proposição comum R, obtém-se:

$$(P \vee Q) \wedge R$$

Demonstração por Tabela-Verdade (Equivalência Lógica):

A tabela-verdade abaixo comprova que as duas expressões são semanticamente idênticas para todas as 8 combinações possíveis, caracterizando uma **Tautologia** na relação bicondicional:

P	Q	R	$P \wedge R$	$Q \wedge R$	$(P \wedge R) \vee (Q \wedge R)$	$P \vee Q$	$(P \vee Q) \wedge R$	Equivalência ($\leftrightarrow$)
V	V	V	V	V	V	V	V	V
V	V	F	F	F	F	V	F	V
V	F	V	V	F	V	V	V	V
V	F	F	F	F	F	V	F	V
F	V	V	F	V	V	V	V	V
F	V	F	F	F	F	V	F	V
F	F	V	F	F	F	F	F	V
F	F	F	F	F	F	F	F	V

Conclusão: Como todos os valores da coluna de equivalência são Verdadeiros (Tautologia), fica formalmente comprovado que a simplificação mantém 100% da integridade lógica do jogo, com redução de código e maior clareza arquitetural.